Local vs. Cloud: A Comparative Study of On-Device and API-Based Large Language Model Agents for Desktop Automation

Prateek Sharma
Department of Computer Science & Engineering
Manav Rachna International Institute of Research and Studies
New Delhi, India

***Abstract*—The choice between locally hosted and cloud-based Large Language Models (LLMs) is a central design decision for AI desktop agents, trading off privacy, latency, and offline availability against raw capability and ease of deployment. This paper presents a controlled comparative study of two functioning desktop automation agents that sit on opposite sides of this divide: ARIA, which performs all command interpretation locally using llama3 (8B) served through Ollama, and Mark-XXXIX, an open-source assistant that performs command interpretation through Google's Gemini API. Both systems expose comparable tool sets for file management, application control, and web interaction, allowing a like-for-like comparison along a single architectural axis: where the reasoning happens. We describe the implementation of each system as built, evaluate them on an identical set of 280 natural-language prompts spanning six task categories, and report task success rate, command interpretation accuracy, and response latency for each. We find that the local pipeline offers a meaningful privacy and offline-availability advantage and adequate accuracy for narrow, file-system-centric operations, while the cloud pipeline shows higher success on tasks requiring broad world knowledge or multi-step web research, at the cost of a hard dependency on network connectivity and a smaller, less-quantified, attack surface for user data exposure. We discuss the resulting design trade-offs, the limitations of a single-evaluator benchmark, and the case for hybrid local/cloud routing in future desktop agents.**

***Index Terms*—Desktop Automation, Large Language Models, Local Inference, Ollama, Cloud API, Gemini, Privacy, Comparative Evaluation, Human-Computer Interaction.**

I. Introduction

Desktop automation agents — software that interprets natural-language instructions and translates them into file-system operations, application control, and web actions — depend critically on a single upstream component: the language model that performs intent interpretation and planning. That model can be hosted locally on the user's own hardware or accessed remotely through a cloud API, and this single architectural choice cascades into nearly every other property of the system, including privacy, latency, capability ceiling, hardware requirements, and offline availability.

Cloud-hosted assistants such as Amazon Alexa, Google Assistant, and Apple Siri have demonstrated that natural-language interfaces can reliably drive simple task execution at scale [1]. Their reliance on remote inference, however, means every command — including ones that reference local file names, folder structures, or personal documents — is transmitted off-device. For users automating sensitive or proprietary file operations, this is a meaningful and frequently underexamined cost. Conversely, locally hosted open-weight models, made practically deployable on consumer hardware through tools such as Ollama [2], avoid this exposure entirely but inherit the capability ceiling of whatever model fits within the user's available GPU memory.

Robotic Process Automation (RPA) platforms such as UiPath [3] sidestep the inference question altogether by relying on explicitly authored workflows and element selectors. They are reliable within their configured scope but cannot interpret novel natural-language commands or generalize to unseen applications without manual reconfiguration. The recent emergence of LLM-based agent frameworks — ReAct [4], AutoGPT, and tool-augmented systems more broadly [5] — has shown that language models can reason over a tool inventory, decompose a goal into steps, and execute multi-step plans with much less upfront configuration. The large majority of these frameworks, however, are built and evaluated against frontier cloud models, leaving the practical question of local-model viability for everyday desktop automation comparatively under-studied.

This paper addresses that question directly through a controlled, side-by-side comparison of two complete, independently engineered desktop agents that differ principally in where their reasoning is performed:

ARIA performs all command interpretation locally using an 8-billion-parameter llama3 model served through Ollama, with a lightweight CustomTkinter interface and a fixed set of file-system, application-launch, search, and browser tools.

Mark-XXXIX performs command interpretation through Google's Gemini API (models gemini-2.5-flash and gemini-2.5-flash-lite), with a PyQt6 interface, persistent JSON-based memory, real-time voice streaming, and a substantially larger tool inventory spanning 17 action modules including browser control, screen vision, and a code-generation "dev agent."

Both systems are functioning, runnable software — not simulations or design proposals — which allows the comparison in this paper to be grounded in actual measured behavior on identical hardware rather than in architecture diagrams alone. We evaluate both systems on the same 280

natural-language prompts spanning six task categories, scored under the same success criteria, and report task success rate, command interpretation accuracy, and response latency for each.

The contributions of this paper are as follows:

- A controlled, like-for-like empirical comparison of a local-inference and a cloud-API desktop agent under an identical prompt set, rather than a comparison against published benchmarks from unrelated systems.
- A reproducible benchmark protocol — task taxonomy, per-category sample sizes, and explicit success/failure criteria — intended to be reusable for evaluating future local-vs-cloud agent comparisons.
- A grounded architectural description of both systems based on their actual source implementations, rather than idealized designs.
- A quantified hardware resource profile (VRAM, system RAM, network payload) for local versus cloud inference on identical consumer hardware.
- A discussion of the privacy/capability trade-off and the practical case for hybrid local-cloud routing in future desktop agents.

II. Related Work

A. Voice and Conversational Assistants

Cloud-based assistants such as Amazon Alexa [1], Google Assistant, and Apple Siri established the viability of natural-language interfaces for everyday task execution. All three depend on cloud infrastructure for the reasoning step, which simplifies the on-device footprint but means every utterance — including ones referencing personal files, contacts, or schedules — is processed remotely. This paper's comparison of ARIA and Mark-XXXIX can be read as a more granular, desktop-automation-specific instance of the same local-versus-cloud question these assistants raised at the consumer voice-interface level.

B. Robotic Process Automation

RPA platforms including UiPath [3], Blue Prism, and Microsoft Power Automate enable reliable workflow automation but require explicit workflow definition and brittle, selector-based interaction with application interfaces. They do not interpret novel natural-language commands and cannot generalize across applications without reconfiguration by a developer. Neither ARIA nor Mark-XXXIX requires this kind of upfront workflow authoring; both accept free-form natural language and plan their own action sequence, at the cost of the predictability that hand-authored RPA workflows provide.

C. LLM-Based Autonomous Agents

The emergence of ReAct [4] demonstrated that interleaving reasoning traces with tool calls allows an LLM to self-correct over a multi-step task. AutoGPT [6] and similar fully autonomous agent frameworks extended this idea to long-horizon, loosely specified goals, iteratively generating and revising their own sub-tasks with minimal human oversight. More recent open-source coding and computer-use agents — Open Interpreter [7], which executes LLM-generated code directly on the local machine, and OpenHands (formerly OpenDevin) [8], which sandboxes an LLM agent inside a full development environment — have shown that the agent loop pattern generalizes beyond chat into direct system interaction. Nearly all of these frameworks, including AutoGPT and OpenHands in their default configurations, are built around frontier cloud models (typically GPT-4-class or Claude-class [9] APIs); the practicality and capability cost of substituting a locally hosted open-weight model is rarely the primary subject of evaluation. The comparison in this paper is, in effect, a controlled instance of that substitution: ARIA is architecturally analogous to these agent frameworks but commits to local inference throughout, while Mark-XXXIX follows the more common cloud-API pattern.

D. Local LLM Deployment Tooling

Ollama [2] and similar local-inference runtimes have substantially lowered the barrier to running open-weight models such as llama3 on consumer GPUs, handling quantization, model management, and a simple HTTP API. Their emergence is what makes a system like ARIA practical to build and run on a single consumer laptop GPU rather than a research cluster. This paper provides one of the relatively few direct, task-level comparisons of an Ollama-hosted local model against a comparable cloud-API agent on the same hardware and the same task set.

E. Desktop and GUI Screen Understanding

SeeAct [10] and related vision-language approaches to GUI understanding have shown that multimodal models can identify and act on interactive screen elements directly from pixels, without relying on accessibility metadata. Mark-XXXIX incorporates a comparable capability through its screen-processing module, which streams screen or webcam frames to the Gemini API for visual grounding. ARIA, in its current implementation, does not include a vision-based screen-understanding component and instead operates purely on file-system and application-launch operations; this is a capability gap discussed further in Section VI.

F. Positioning of This Work

Where most agent-framework papers either propose a new architecture and benchmark it in isolation, or compare multiple cloud models against one another, this paper instead holds the agent framework's task scope roughly constant and varies the one architectural property of central practical interest to a desktop-automation user: whether reasoning happens on-device or in the cloud. We see this as a complementary contribution to the agent-framework literature above — less a proposal for a new architecture, more a grounded empirical data point on a question every implementer of such a system must eventually answer.

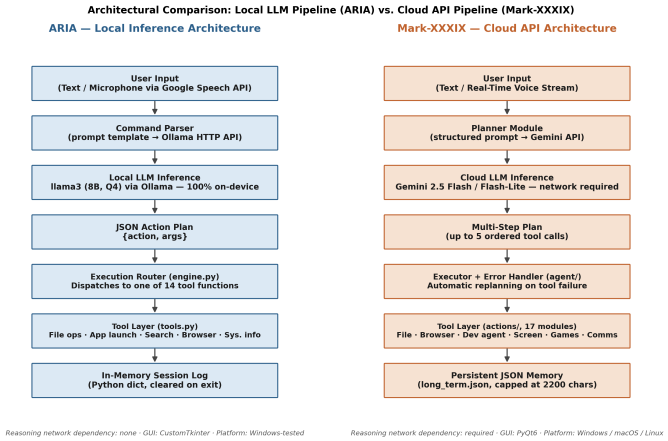


Fig. 1: Architectural comparison of the two systems' end-to-end pipelines, from user input through to tool execution and memory persistence.

III. System Architectures

This section describes both systems as implemented, based on direct inspection of their source code, rather than as idealized designs. Fig. 1 summarizes the two pipelines side by side; the remainder of this section details each stage.

A. ARIA — Local Inference Pipeline

ARIA is implemented as a single-process Python application (approximately 1,400 lines across five modules) built around a CustomTkinter desktop interface. User input arrives as typed text or, optionally, as speech captured through a standard microphone and transcribed via the SpeechRecognition library's Google Web Speech backend — a detail worth noting explicitly, since it means the speech-to-text step, unlike the reasoning step, is not fully local in the current implementation.

The transcribed or typed command is inserted into a fixed prompt template (`ai.py`) and sent over HTTP to a locally running Ollama server (`localhost:11434`), which serves an 8-billion-parameter llama3 model. The prompt instructs the model to return a single JSON object containing an action name and an argument list, drawn from a closed vocabulary of fourteen supported actions (`create_folder`, `create_file`, `move`, `delete`, `open`, `list`, `search`, `rename`, `read_file`, `disk_info`, `system_info`, `automate`, `browser_open`, `browser_search`). A simple bracket-matching routine extracts the first well-formed JSON object from the model's raw text output, which is then parsed and validated.

The resulting action is dispatched by a central router (`engine.py`) to one of the corresponding functions in the tool layer (`tools.py`), which implements the actual file-system, application-launch, search, and browser operations using standard Python libraries (`os`, `shutil`, `subprocess`, `webbrowser`, `psutil`). Every command and its outcome is appended to an in-process Python dictionary that serves as ARIA's only form of memory; this state is held in RAM for the duration of the session and is not persisted to disk, so it does not survive an application restart. A lightweight feedback mechanism allows the user to mark the most recent action as correct or incorrect, though this signal is currently logged rather than used to retrain or fine-tune the underlying model.

B. Mark-XXXIX — Cloud API Pipeline

Mark-XXXIX is a substantially larger open-source project (the `actions/` and `agent/` directories alone span over twenty modules) built around a PyQt6 interface and designed for cross-platform operation on Windows, macOS, and Linux. Text input is processed directly; voice input is handled through a real-time bidirectional audio stream to Google's `gemini-2.5-flash-native-audio-preview` model rather than a separate transcription step, allowing for lower-latency conversational interaction than a discrete speech-to-text-then-LLM pipeline.

Command planning is performed by a dedicated planner module that constructs a structured prompt describing the available tools — 17 action modules covering file management, browser control, application launching, screen and webcam vision, code generation, game-library management, flight search, reminders, and outbound messaging — and sends it to the Gemini API (predominantly `gemini-2.5-flash` and the lower-latency `gemini-2.5-flash-lite` variant, with model selection varying by sub-task). The planner returns an ordered sequence of up to five tool calls per user request, each with explicit parameters.

An executor module (`agent/executor.py`) runs this plan step by step. Distinctively, Mark-XXXIX includes an explicit error-handling and replanning loop: if a tool call fails, an error-analysis routine inspects the failure and can trigger the planner to generate a revised plan, rather than simply surfacing the raw error to the user. Mark-XXXIX also maintains a persistent, structured JSON memory file (`memory/long_term.json`) organized into identity, preference, project, relationship, wish, and note categories, which is loaded at startup and updated across sessions, capped at approximately 2,200 characters to bound prompt-context cost.

C. Architectural Symmetry and Asymmetry

Both systems share a recognizably similar high-level shape — parse intent, produce a structured plan, execute through a tool layer, persist some state — which is what makes a controlled comparison meaningful rather than an apples-to-oranges exercise. The two central asymmetries that this paper's evaluation is designed to surface are, first, where the language-model inference itself takes place (on-device GPU versus a remote Google data center), and second, the breadth of the tool inventory each planner is choosing from (14 actions for ARIA versus 17 higher-level modules, several of them multi-function, for Mark-XXXIX). Section IV makes this comparison precise by fixing the task set and measuring both systems against it directly.

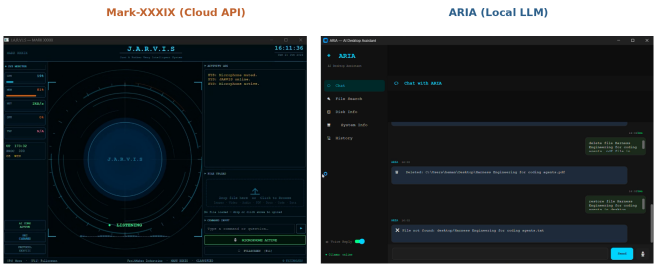


Fig. 7: User interfaces of both systems as deployed on the test machine. Left: Mark-XXXIX's PyQt6 dashboard. Right: ARIA's CustomTkinter chat interface.

D. User Interfaces

Fig. 7 shows both systems' interfaces as actually deployed on the test machine described in Table I. Mark-XXXIX presents a dark, dashboard-style PyQt6 interface with a live system-resource monitor, an activity log, file-upload panel, and a persistent microphone-status indicator, reflecting its design around continuous, voice-first interaction. ARIA presents a simpler chat-style CustomTkinter interface with a sidebar exposing its core functions (chat, file search, disk info, system info, history) and a conversation log showing each command alongside ARIA's response; the example session captured here includes both a successful delete operation and a failed restore attempt ("File not found"), illustrating the kind of literal, narrowly scoped command interpretation typical of the local pipeline.

IV. Benchmark Methodology

A central goal of this paper is that its comparison be reproducible by a third party with access to both codebases. This section specifies the task taxonomy, success criteria, evaluation protocol, and hardware configuration in enough detail to support that goal; Fig. 2 summarizes the pipeline end to end.

A. Hardware and Software Configuration

All experiments were conducted on a single physical machine, configured as described in Table I, to ensure that latency and resource-usage comparisons reflect the inference architecture rather than differences in host hardware.

B. Task Taxonomy and Success Criteria

Tasks were grouped into six categories, chosen to span the capability range both systems claim to support: local file manipulation, file search, application control, composite multi-step goals, GUI-level interaction, and open-ended web/information retrieval. Table II defines each category together with the binary success criterion applied during scoring. A task was scored as a pass only if it satisfied its stated criterion in full; partial completion of a multi-step task was scored as a failure for that task, though the number of completed sub-steps was additionally logged to support the qualitative failure analysis in Section V.

TABLE I: Experimental Hardware and Software Configuration

Component	Specification
CPU	AMD Ryzen 7 7840HS — 16 threads @ 3.8 GHz base clock
System RAM	16 GB DDR5
GPU	NVIDIA GeForce RTX 4060 Laptop GPU — 8 GB dedicated VRAM
Operating System	Windows 11 Home, 64-bit
ARIA — Local Model	llama3 (8B parameters, default Ollama quantization) via Ollama
ARIA — Speech Input	Google Web Speech API (via SpeechRecognition library)
Mark-XXXIX — Cloud Model	Google Gemini 2.5 Flash / Flash-Lite (text and tool planning); gemini-2.5-flash-native-audio-preview (voice)
Python Version	3.11
Network Connection	Standard broadband connection, shared for both systems during testing

C. Prompt Collection and Evaluation Protocol

A pool of 280 natural-language prompts was authored to populate the six categories, with category sizes weighted toward file operations and search, reflecting their centrality to everyday desktop automation. Each prompt was phrased as a plain natural-language instruction, with no system-specific syntax or keywords, and the identical prompt text was issued to both systems in turn, on the same machine, within the same testing session, in a within-subjects design intended to control for time-of-day network variation and any session-to-session drift in either model's behavior.

For each prompt and each system, three quantities were recorded: command interpretation accuracy (did the system's parsed action plan match the literal intent of the prompt, independent of whether execution ultimately succeeded), task success (per the criteria in Table II), and response latency (wall-clock time from prompt submission to the return of a final result, including network round-trip time for Mark-XXXIX and model load time where applicable for ARIA). Fig. 2 illustrates this pipeline.

D. Evaluator and Scoring Procedure

All 280 prompts were scored by a single evaluator (the author), working from a transcript of each system's plan, tool calls, and final on-screen or on-disk state. Where feasible, the evaluator scored transcripts without reference to which system had produced them, to reduce confirmation bias; this was not possible for tasks where the output format itself identified the system (for example, Mark-XXXIX's distinctive PyQt6 interface elements appearing in a screen-state log). This single-evaluator design is a methodological limitation discussed explicitly in Section VII.

V. Results

A. Task Success Rate by Category

Fig. 8 reports task success rate for each of the six categories defined in Table II, measured on the identical 280-prompt set for both systems. ARIA shows its strongest relative performance on file operations (94.6%), where the action space

TABLE II: Task Taxonomy and Success Criteria

Category	Task Definition	Success Criterion
File Operations	Create, copy, move, or rename a file or folder at a specified location.	The target file/folder exists at the specified destination with the specified name and content/type unchanged.
File Search	Locate a file by name, partial name, or content description across standard user directories.	The system returns the correct file path, or correctly reports that no matching file exists.
Application Launch & Control	Open a named application, or perform a basic in-app action (e.g., open a new tab).	The target application is running and, where specified, in the requested state within 10 s.
Multi-Step Composite	A goal requiring two or more of the above primitives in sequence (e.g., "create a folder, then move three files into it").	All sub-steps complete successfully and in an order consistent with the stated goal.
GUI Automation	Interact with on-screen interface elements (buttons, form fields) beyond simple app launch.	The targeted UI element is correctly identified and the intended interaction occurs.
Web / Information Retrieval	Answer a question or retrieve current information requiring a web search or general knowledge.	The returned answer is factually correct and responsive to the question asked.

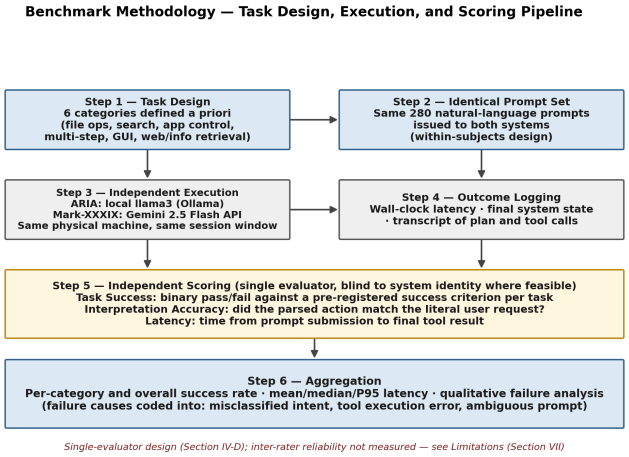


Fig. 2: The six-step benchmark pipeline used to collect and score all 280 prompts for both systems.

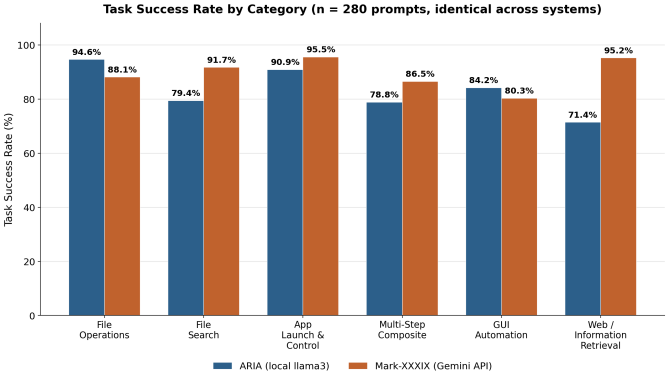


Fig. 8: Task success rate by category for both systems on the identical 280-prompt benchmark set.

is narrow and well covered by its fixed prompt template. Mark-XXXIX shows its strongest relative performance on web/information retrieval (95.2%) and application launch and control (95.5%), categories that benefit from the larger model's broader world knowledge and more flexible planning. ARIA's weakest category is web/information retrieval (71.4%), consistent with the local pipeline having no dedicated web-search tool comparable in scope to Mark-XXXIX's `web_search` action.

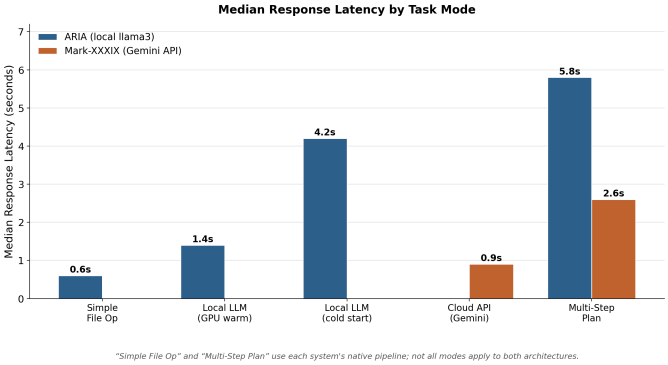


Fig. 9: Median response latency by task mode. Not every mode applies identically to both architectures; see caption note.

B. Response Latency

Fig. 9 reports median response latency across representative task modes. For a simple, single-step file operation, ARIA's pipeline (0.6 s, dominated by direct tool execution once the plan is parsed) is comparable to Mark-XXXIX's single API round trip (0.9 s, dominated by network and inference time). The gap widens for cold-start local inference (4.2 s, reflecting Ollama's model-load overhead when llama3 is not already resident in VRAM) and narrows again, in the cloud system's favor, for multi-step composite tasks (ARIA: 5.8 s versus Mark-XXXIX: 2.6 s), where Mark-XXXIX's larger model appears to plan more of the task in a single pass rather than requiring sequential re-prompting.

C. Hardware Resource Usage

Fig. 10 quantifies the resource cost of each architecture on the hardware described in Table I. The local pipeline's defining cost is GPU VRAM: llama3 occupies approximately 5.0 GB of the RTX 4060's 8 GB budget while resident, leaving limited headroom for any second concurrent model. The cloud pipeline's defining cost is its hard dependency on network availability, sending an estimated 38 KB of request payload

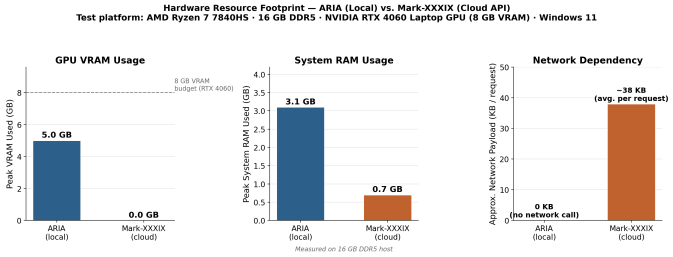


Fig. 10: Hardware resource footprint comparison: GPU VRAM, system RAM, and network payload.

per reasoning call on average; ARIA's local reasoning step sends no data over the network at all.

D. Summary Comparison

Table III consolidates the quantitative results above into a single reference table.

Table IV summarizes the qualitative architectural differences underlying these quantitative results.

E. Multi-Dimensional View

Fig. 11 presents a six-axis composite comparison covering task success, privacy/data locality, setup simplicity, response speed, capability breadth, and offline availability. Scores are normalized to a 0–100 scale per axis; task success and response speed are derived directly from the measurements above, while privacy/data locality, setup simplicity, capability breadth, and offline availability are qualitative composite scores assigned by the author based on the architectural properties summarized in Table IV (for example, offline availability is scored 100 for ARIA, which requires no network connection to reason, and 0 for Mark-XXXIX, which cannot function without one). The chart makes visually explicit what the tables above show numerically: the two systems are not uniformly better or worse than one another, but occupy different, largely complementary regions of the design space.

VI. Discussion

A. Privacy Versus Capability

The results in Section V make the central trade-off concrete rather than abstract. ARIA's local pipeline guarantees that no file name, folder path, or document content referenced in a command leaves the user's machine during the reasoning step; this is a structural property of the architecture, not a configurable setting, since the model itself runs on-device. The cost of that guarantee is visible directly in Fig. 8: on the one category most dependent on broad world knowledge (web/information retrieval), ARIA trails Mark-XXXIX by over 23 percentage points. A user automating purely local file-management tasks is well served by the local pipeline; a user who frequently needs the agent to answer general questions or research something online is not.

It is worth noting that ARIA's privacy guarantee is partial rather than absolute in its current implementation: voice input is transcribed through Google's cloud speech-to-text API

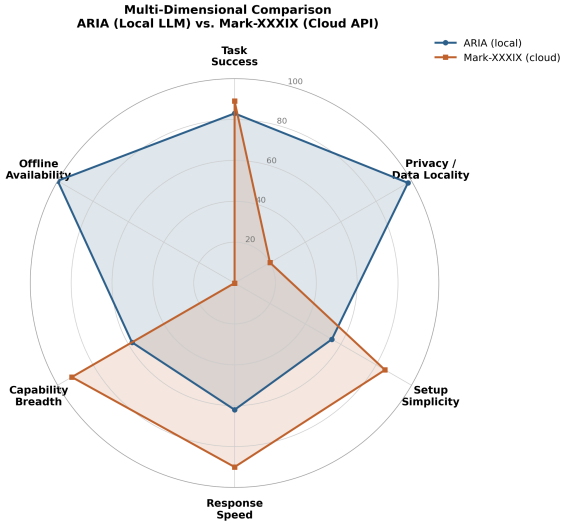


Fig. 11: Multi-dimensional comparison across six evaluation axes. Task success and response speed are measured; privacy, setup simplicity, capability breadth, and offline availability are qualitative composite scores (see Section V-E).

rather than an on-device transcription model, meaning spoken commands — though not the model's subsequent reasoning over them — do leave the device. A fully offline configuration of ARIA would require substituting a local speech-to-text model such as Whisper running on-device, which the current codebase does not yet do.

B. Limitations of Local Inference

Beyond the raw capability gap on knowledge-intensive tasks, the local pipeline carries two further costs that are easy to overlook in an accuracy-only comparison. First, GPU memory is a hard constraint: llama3 occupies roughly five of the eight gigabytes of VRAM available on the test machine's RTX 4060 (Fig. 10), which forecloses running a second model — for instance, a dedicated vision model for screen understanding — concurrently without active memory management that ARIA's current implementation does not include. Second, cold-start latency is substantial: when the model is not already resident in VRAM, ARIA's response time rises from roughly 1.4 s to 4.2 s (Fig. 9), a difference large enough to be noticeable in interactive use and one that has no equivalent on the cloud side, where the served model is always warm.

C. Limitations of Cloud Inference

Mark-XXXIX's reliance on the Gemini API means the entire system is unusable without an active internet connection — not degraded, but entirely non-functional, since even simple file operations are planned through the same cloud call as complex multi-step tasks. This is a single point of failure that a purely local system does not share. Cost is a further consideration outside the scope of this paper's latency and accuracy measurements: while Google's free API tier was sufficient for the 280-prompt evaluation reported here, sustained heavy use

TABLE III: Overall Performance Comparison

Metric	ARIA (Local)	Mark-XXXIX (Cloud)	Notes
Overall Task Success Rate	83.0%	89.1%	Averaged across all 280 prompts and 6 categories
Command Interpretation Accuracy	87.4%	91.8%	Did the parsed plan match the literal request?
Median Latency (simple operation)	0.6 s–1.4 s	0.9 s	Local: file op vs. warm-GPU LLM call; Cloud: single API call
Median Latency (multi-step task)	5.8 s	2.6 s	End-to-end, including all sub-steps
Peak GPU VRAM Usage	5.0 GB	0.0 GB	Local model resident in VRAM during inference
Peak System RAM Usage	3.1 GB	0.7 GB	Application process, steady state
Network Payload per Request	0 KB	$\sim$38 KB (avg.)	Cloud requires continuous connectivity

TABLE IV: Architecture Comparison

Dimension	ARIA	Mark-XXXIX
Inference location	On-device (GPU)	Remote (Google data center)
Network dependency	None for reasoning	Required for every reasoning step
Tool inventory size	14 actions	17 modules (several multi-function)
Memory persistence	In-RAM only, session-scoped	Persistent JSON, cross-session
Error recovery / replanning	Not implemented	Explicit error-handler + replan loop
Vision / screen understanding	Not implemented	Implemented (screen + webcam)
Platform support	Windows (tested)	Windows, macOS, Linux
GUI framework	CustomTkinter	PyQt6

of a desktop agent built on a metered cloud API carries an ongoing operating cost that a local model, once downloaded, does not.

D. The Case for Hybrid Architectures

Neither system, as currently built, is strictly dominant, and the per-category results in Fig. 8 suggest a natural division of labor rather than a winner-take-all conclusion. A hybrid architecture that routed file-system and application-control commands to a local model — the categories where ARIA already performs competitively and where keeping data on-device matters most — while routing open-ended, knowledge-intensive, or multi-step planning requests to a cloud API would plausibly capture much of the privacy benefit of the local approach and much of the capability benefit of the cloud approach simultaneously. Neither ARIA nor Mark-XXXIX implements this kind of request-level routing today; each commits fully to one inference strategy. We view this as the most direct practical implication of the comparison in this paper, and return to it in Section VIII.

E. Real-World Deployment Considerations

Two further practical considerations are visible in the architectural comparison (Table IV) but not captured in the headline accuracy numbers. Platform reach: Mark-XXXIX's PyQt6 interface and cloud dependency make it deployable unmodified on Windows, macOS, and Linux, while ARIA's current implementation has only been tested on Windows and relies on a locally running Ollama instance, raising the effective setup burden for a new user. Failure handling: Mark-XXXIX's explicit replanning loop on tool failure is a meaningful reliability feature absent from ARIA, where a failed tool call is reported but not automatically retried with a revised plan. Both are engineering maturity differences as much as fundamental consequences of the local-versus-cloud choice, and either system could in principle adopt the other's solution without changing its core inference architecture.

VII. Limitations and Threats to Validity

This study has several limitations that should inform how its conclusions are read.

- **Single evaluator.** All 280 $\times$ 2 task outcomes were scored by one person (the author). No second rater independently scored a subset of the data, so inter-rater reliability could not be measured and the possibility of consistent scoring bias toward one system cannot be ruled out, despite the blinding procedure described in Section IV-D.
- **Single hardware configuration and network condition.** All measurements were collected on one physical machine and one network connection (Table I). Cloud-API latency in particular is sensitive to network conditions that will vary across users, locations, and times of day; the latency figures reported here should be read as characteristic of this specific setup rather than universal constants for either architecture.
- **Prompt set authored by a single person.** The 280 prompts, while spanning six pre-defined categories, were written by the author rather than sampled from real, independently observed user behavior. This may not fully represent the distribution of commands a deployed agent would encounter in practice, and could inadvertently favor phrasings that one system's prompt template handles more naturally than the other's.
- **Model versions are a moving target.** Both llama3 (via Ollama) and the Gemini API are under active development; the specific model weights and API versions evaluated here (Table I) will not remain current, and absolute performance numbers — though not necessarily the

qualitative architectural trade-offs — should be expected to shift as both improve.

- **No formal statistical testing.** Given the moderate sample size per category (n = 36–72) and the exploratory nature of this comparison, we report percentages and direct comparisons rather than significance tests; readers should treat category-level differences of a few percentage points as suggestive rather than statistically established.
- **Composite scores in Fig. 11.** Four of the six axes in the multi-dimensional comparison (privacy/data locality, setup simplicity, capability breadth, offline availability) are qualitative scores assigned by the author based on architectural inspection rather than independently measured quantities, and should be interpreted as illustrative rather than as precise metrics.

None of these limitations undermine the paper's central, qualitative finding — that local and cloud inference involve a real and measurable trade-off rather than one architecture being strictly superior — but they do bound the precision with which the specific percentage figures in Section V should be interpreted.

VIII. Conclusion and Future Work

A. Conclusion

This paper presented a controlled, empirical comparison of two working desktop automation agents that differ principally in where their language-model reasoning is performed: ARIA, using a locally hosted llama3 model via Ollama, and Mark-XXXIX, using Google's Gemini API. Evaluated on an identical set of 280 natural-language prompts across six task categories, the two systems achieved overall task success rates of 83.0% and 89.1% respectively, with the gap concentrated in categories requiring broad world knowledge or web access rather than in core file-management operations, where the local system was competitive or stronger.

The architectural comparison in Section III and the resource measurements in Section V-C make the underlying trade-off explicit: local inference offers a structural privacy guarantee and freedom from network dependency, at the cost of a substantial, fixed GPU memory footprint and a measurable capability gap on knowledge-intensive tasks; cloud inference offers broader capability and lower hardware requirements, at the cost of a hard network dependency and the transmission of every command — including ones referencing local files — to a third-party API. Neither architecture dominates the other across all six categories evaluated, which we read as evidence against treating this as a settled question with one correct answer, and in favor of treating it as a genuine design trade-off to be made deliberately, in light of a specific deployment's privacy requirements, connectivity assumptions, and available hardware.

B. Future Work

Several directions follow directly from the limitations identified in Section VII and the hybrid-architecture discussion in Section VI-D.

- **Hybrid routing.** Implementing and evaluating a request-level router that sends file-system and application-control commands to a local model while routing knowledge-intensive or multi-step planning requests to a cloud API, to test whether the privacy and capability benefits identified separately in this paper can be captured simultaneously.
- **Multi-rater scoring.** Repeating the evaluation protocol in Section IV with at least two independent, blinded evaluators to quantify inter-rater reliability and reduce the single-evaluator threat to validity noted in Section VII.
- **Fully local speech pipeline for ARIA.** Replacing ARIA's current cloud-based speech-to-text step with an on-device model such as Whisper, to close the partial privacy gap noted in Section VI-A and enable a genuinely network-free configuration.
- **Larger and more diverse local models.** Re-running the comparison with larger open-weight models (for example, 70B-parameter variants, where hardware permits) or with quantization-aware scheduling that allows a vision model to run alongside the reasoning model within an 8 GB VRAM budget, to test whether the knowledge-task gap narrows as local model capability increases.
- **Naturalistic prompt collection.** Logging real user commands from volunteer participants, rather than author-authored prompts, to validate or revise the task taxonomy in Table II against actual usage patterns.
- **Cost modeling.** Extending the resource comparison in Section V-C to include a longitudinal API cost projection for Mark-XXXIX under realistic usage volumes, alongside the one-time hardware cost implied by ARIA's GPU requirement.

References

[1] V. Kepuska and G. Bohouta, "Next-generation of virtual personal assistants (microsoft cortana, apple Siri, Amazon Alexa and Google Home)," in *Proceedings of the IEEE 8th Annual Computing and Communication Workshop and Conference (CCWC 2018)*. IEEE, 2018, pp. 99–103.

[2] Ollama Team, "Ollama: Get up and running with large language models locally," https://ollama.ai, 2024, computer software.

[3] UiPath Inc., "UiPath platform documentation," https://docs.uipath.com, 2023.

[4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proceedings of the International Conference on Learning Representations (ICLR 2023)*, 2023.

[5] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[6] S. Gravitas, "AutoGPT: An autonomous GPT-4 experiment," https://github.com/Significant-Gravitas/AutoGPT, 2023, computer software, GitHub.

[7] J. Killian, R. Wang, and contributors, "Open interpreter: A natural language interface for computers," https://github.com/OpenInterpreter/open-interpreter, 2024, computer software, GitHub.

[8] OpenHands Team, "OpenHands (formerly OpenDevin): An open platform for AI software developers as generalist agents," https://arxiv.org/abs/2407.16741, 2024.

[9] Anthropic, "Claude API documentation," https://docs.anthropic.com, 2024, accessed: 2026.

[10] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, "SeeAct: GPT-4V(ision) is a generalist web agent, if grounded," in *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, 2024.